BeeFast Delivery System

Graduation Project 1 - Report, Part One

Faculty of Artificial Intelligence Engineering - Syrian Private University
Student: [fajer abo trabi] , [ashraf badour] Supervisor: [Eng. Shadi Blaidy]

1. Scientific Problem

In many local markets, ordering from stores, restaurants and cafes still depends on phone calls or scattered social-media channels. This causes four main problems:

- Customers cannot browse products or compare stores in one place.
- Orders are handled manually, which leads to errors and lost requests.
- Customers cannot track their order after it is placed.
- Store owners and administrators have no central way to assign drivers, monitor order status or manage stock.

The scientific problem is therefore how to design and implement an integrated web-based delivery management system that connects customers, administrators and drivers through role-based workflows, order tracking on a map and automatic notifications.

2. Project Objectives

2.1 General objective

To build BeeFast, a web-based delivery system that automates the full order cycle, from browsing stores to delivering the order to the customer.

2.2 Specific objectives

1. Provide secure registration and login with three roles: user, driver and admin.
2. Allow administrators to manage stores and products (add, edit and delete).
3. Let customers browse stores by type, search by name, and manage a cart and a favorites list, with stock updated automatically.
4. Support order placement with delivery location selection on an interactive map (Leaflet and OpenStreetMap).
5. Let administrators assign drivers, and let drivers update the order status through the lifecycle pending, in_progress, on_the_way and delivered, with cancellation allowed only while the order is pending.
6. Send automatic notifications at every status change.
7. Apply a loyalty discount of 5% per previous order, capped at 20% of the subtotal.

3. State of the Art

Online ordering and delivery is served today by three broad families of solutions. This section compares them with BeeFast. The comparison is qualitative and describes typical capabilities of each family; references will be added in the final report.

3.1 Informal ordering by phone or messaging apps

The most common approach in small local markets. It needs no technology investment, but orders are written down manually, there is no product catalogue, no tracking, and no central record of orders, drivers or stock.

3.2 Large delivery platforms

Platforms such as Talabat, Uber Eats and Deliveroo offer a catalogue, live tracking, ratings and online payment. They are powerful, but they are built for large markets: they are owned and controlled by the platform company, they charge commissions to stores, and they are not easy to adapt or deploy in a market where such services are not available or not suitable.

3.3 Generic e-commerce templates

Ready-made e-commerce systems provide a catalogue and a cart, but they are designed for shipping goods, not for local delivery. They usually lack driver roles, order status workflows tied to drivers, and map-based delivery location.

3.4 Comparison

Feature	Phone / WhatsApp ordering	Large delivery platforms	Generic e-commerce templates	BeeFast
Browse stores and products in one place	No	Yes	Partly	Yes
Order tracking on a map	No	Yes	Rarely	Yes (Leaflet / OSM)
Admin-controlled driver assignment	Manual	Automated, platform-owned	No	Yes (admin assigns)
Role-based workflow (user / driver / admin)	No	Yes	Usually two roles	Yes, three roles
Automatic status notifications	No	Yes	Email only	Yes (in-app)
Loyalty discount	No	Varies	Plugin	Yes (5% per order, max 20%)
Adaptable to a small local market	Yes	Limited	Yes	Yes

Table 1. Comparison between existing approaches and BeeFast

3.5 Position of BeeFast

BeeFast combines the strongest features of the delivery platforms (catalogue, role-based workflow, map-based location and notifications) in a lightweight system that is built on open-source technologies, is fully controlled by the administrator, and can be adapted to a local market at low cost.

4. Solution Idea

BeeFast is a three-layer web application built with the Laravel framework following the MVC pattern. Users access it through a web browser. Requests go through an authentication and role-based access layer, then to controllers that handle stores, products, the cart, orders and favorites. Data is stored in a MySQL database through Eloquent ORM, and product and store images are kept in file storage. The delivery location is selected and displayed using Leaflet with OpenStreetMap, and a notification module records a message for the relevant user whenever an order changes state.

4.1 System architecture (block diagram)

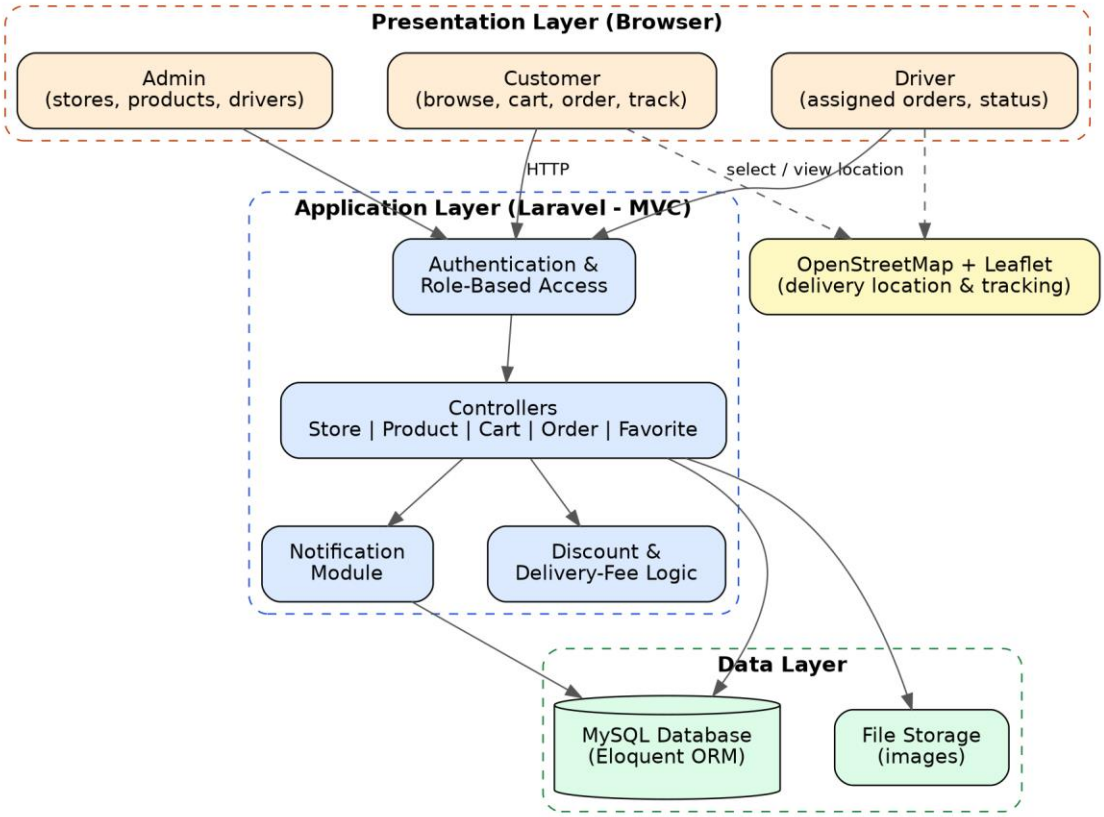


Figure 1. BeeFast system architecture

4.2 Order lifecycle (flow chart)

Figure 2 shows how an order moves through the system. An order is created as pending, then the administrator either accepts it by assigning a driver or rejects it. The driver then updates the status until delivery. The customer may cancel only while the order is still pending.

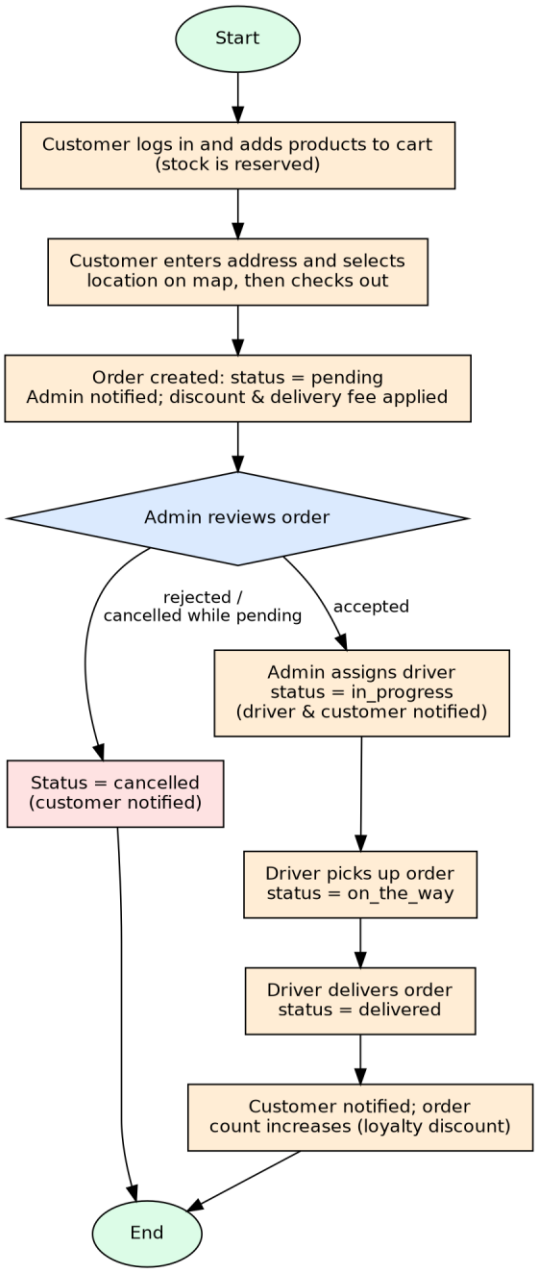


Figure 2. Order lifecycle flow chart

4.3 Actors and responsibilities

- Customer: browses stores and products, manages the cart and favorites, places and tracks orders, cancels pending orders.
- Admin: manages stores, products and users, assigns drivers and monitors all orders.
- Driver: views assigned orders and updates their status (on the way, delivered).

5. Chapter One: Theoretical Study

Chapter 1: Introduction

1-1 Project Overview and Background

In light of rapid technological development and the digital transformation the world is

witnessing, on-demand delivery applications have become an integral part of daily life, meeting the growing need of consumers to obtain goods and services easily and quickly. This project aims to develop an integrated and smart delivery system that connects various stores (restaurants, cafes, grocery stores), users, and drivers, providing a seamless and efficient experience for all parties. The system relies on the latest web development technologies and seeks to offer innovative solutions to traditional challenges facing delivery operations.

1-2 Problem Statement and Academic Significance

Traditional delivery systems suffer from several challenges, most notably inefficiency in order management, difficulty in tracking delivery status, and a lack of effective coordination among stores, drivers, and users. These issues lead to delivery delays, a poor user experience, and increased operational costs. This project highlights its academic significance by providing an integrated software solution that addresses these challenges through process automation, intuitive user interfaces, and the application of smart mechanisms to improve delivery efficiency. It also represents an opportunity to apply concepts learned in software engineering and the development of intelligent systems.

1-3 Project Objectives (Main and Sub-Objectives)

This project aims to achieve a set of main and sub-objectives, formulated according to SMART criteria (Specific, Measurable, Achievable, Relevant, and Time-bound):

Main Objectives:

- Develop an integrated web system for order delivery connecting stores, users, and drivers.
- Improve the efficiency of the delivery process and reduce order response time.
- Provide a seamless and intuitive user experience for all parties.

Sub-Objectives:

- Provide an admin dashboard that allows effective management of stores, products, and orders.
- Develop a user interface for drivers that enables them to receive orders and track delivery routes.

- Create a user interface for users that allows browsing stores, adding products to the cart, and placing orders.
- Implement an instant notification system to update order status for all parties.
- Provide a comprehensive admin dashboard to manage users, stores, products, and orders.

Integrate smart features such as categorizing stores based on popularity and suggesting drivers.

1-4 Project Scope, Constraints, and Assumptions

Project Scope: The project includes the development of a Web Application that supports three types of users: regular user (Customer), and driver (Driver), in addition to an admin dashboard where the Administrator manages stores and products. The system covers the entire order lifecycle from browsing products to receiving the order, focusing on specific cities and geographical areas.

Constraints:

- Limited Budget: May affect the use of some paid external services.
- Available Time: The project must be completed within the timeframe specified for the graduation project.
- Human Resources: The project relies on a limited team.
- Technologies Used: The project is limited to using the Laravel framework and Bootstrap interface.

Assumptions:

- Users and drivers have internet access to reach the application.
- Initial data for stores and products is provided for testing purposes.
- Users and drivers have a minimum basic knowledge of using web applications.

1-5 Project Contribution to Knowledge and Field

This project contributes to the field of artificial intelligence engineering and web applications by providing a delivery system based on a modern software architecture (Laravel) and including smart features to improve user experience and operational efficiency. The main contribution is the integration of mechanisms for categorizing stores based on specific criteria, the development of an effective notification system, and the potential for future expansion to integrate more complex machine learning algorithms to optimize delivery routes and demand forecasts. The project also offers a practical model for applying Domain-Driven Design principles in the context of e-commerce applications and delivery services.

1-6 Research Methodology and Scientific Methods Used

This project adopted an iterative and incremental development methodology, inspired by Agile principles, to ensure flexibility and responsiveness to changes. The project was divided into short development cycles (Sprints), where a set of features is planned, designed, implemented, and tested in each cycle. Descriptive and analytical research methods were used to gather information about requirements, analyze similar systems, and evaluate appropriate technologies. Experimental methodology was also relied upon in the development and testing phases to evaluate system performance and continuously improve it.